



Universidad CENFOTEC

Grupo 3

Programacion Orientada a Objetos

Docente: ROMARIO SALAS CERDAS

Portafolio 1

Julio Alberto Salvador Quiros

28 de Junio 2026

1. Justificación de la elección del sistema

Contexto del sistema

Una biblioteca publica administra libros físicos, gestiona usuarios registrados y lleva el control de prestamos activos. Cada día, el personal necesita saber que libros estan disponibles, quienes los tienen prestados y cuando deben ser devueltos.

Este dominio fue elegido porque contiene exactamente los ingredientes que la POO esta disenada para manejar: entidades del mundo real con identidad propia, relaciones naturales entre ellas y estado que debe protegerse de modificaciones accidentales.

Por qué Programación Orientada a Objetos y no programación común?

En un enfoque procedural, el sistema viviria como funciones globales y arreglos paralelos. Funcionaria para algo minimo, pero presentaria problemas concretos:

Situacion	Programación Común vs POO
Agregar un Administrador	Modificar todas las funciones existentes que reciben personas
Proteger disponible en Libro	Cualquier funcion puede cambiarlo; fácil de desincronizar
Relacionar Prestamo con Libro	Variables paralelas sin garantia de coherencia
Escalar (reservas, multas, etc.)	Requiere reestructurar logica central

La POO resuelve cada uno de esos problemas estructuralmente, no con disciplina de codigo: Persona abstracta permite agregar roles sin tocar lo existente; el encapsulamiento de Libro hace imposible cambiar disponible sin pasar por setDisponible(); la composicion en Prestamo garantiza que libro y usuario siempre estaran juntos y coherentes.

2. Aplicacion de los conceptos POO

2.1 Abstraccion

La abstraccion consiste en modelar solo lo que el sistema necesita de una entidad, ignorando detalles irrelevantes. En este proyecto se aplica a traves de la clase abstracta Persona.

Persona captura lo que todo ser humano del sistema tiene en comun: nombre e id. No importa si es usuario o bibliotecario; ambos comparten esos atributos. Sin embargo, Persona no puede instanciarse directamente porque no tiene sentido hablar de 'una persona generica' en una biblioteca.

El metodo getRol() es abstracto: obliga a cada subclase a definir su propio rol, garantizando que ningun objeto Persona quede sin identidad.

Fragmento clave: Persona.java

```
public abstract class Persona {  
  
    // Solo los datos que todo el sistema necesita de cualquier persona  
    private String nombre;  
    private String id;  
  
    // ABSTRACCION: cada subclase define que rol cumple  
    public abstract String getRol();  
}
```

2.2 Encapsulamiento

El encapsulamiento protege el estado interno de un objeto. En este proyecto, todos los atributos son private. El caso mas importante es Libro: el campo disponible controla si el libro puede prestarse, y solo puede modificarse a traves de setDisponible(boolean).

Esto significa que ningun codigo externo puede accidentalmente marcar un libro como disponible sin pasar por la logica del sistema. El error clasico de 'prestar un libro que ya estaba prestado' se vuelve estructuralmente imposible de cometer desde fuera de la clase.

Fragmento clave: Libro.java

```
public class Libro {  
  
    private String titulo;  
    private String autor;  
    private String isbn;  
    private boolean disponible;    // atributo critico, protegido  
  
    // Getter de solo lectura: nadie puede asignar disponible  
    directamente  
    public boolean isDisponible() { return disponible; }
```

```

        // Unico punto de modificacion controlada
        public void setDisponible(boolean disponible) {
            this.disponible = disponible;
        }
    }
}

```

2.3 Herencia

Usuario y Bibliotecario extienden Persona. Ambos heredan nombre, id y los getters sin duplicar código. Cada uno implementa getRol() con su propio valor. Si mañana se necesita un Voluntario, simplemente extiende Persona: no hay que tocar ningún archivo existente.

Fragmento clave: Usuario.java

```

public class Usuario extends Persona {

    public Usuario(String nombre, String id) {
        super(nombre, id);    // delega a Persona
    }

    @Override
    public String getRol() { return "Usuario"; }
}

```

2.4 Composición

La composición expresa que un objeto está formado por otros y que no tiene sentido sin ellos. Hay dos casos en este proyecto:

- Prestamo está compuesto de un Libro y un Usuario. Al crearse el Prestamo, recibe ambos y de inmediato marca el libro como no disponible. Un préstamo sin libro o sin usuario es imposible.
- GestorPrestamos crea su propio Catalogo en el constructor. Si el gestor deja de existir, el catalogo también. Son inseparables.

Fragmento clave: Prestamo.java

```

public class Prestamo {

    private Libro    libro;    // composicion: el prestamo contiene un
                                libro
    private Usuario  usuario;  // composicion: y un usuario
    private String   fecha;

    public Prestamo(Libro libro, Usuario usuario, String fecha) {

```

```

        this.libro    = libro;
        this.usuario = usuario;
        this.fecha    = fecha;
        libro.setDisponible(false); // estado actualizado al nacer el
prestamo
    }
}

```

2.5 Agregacion

La agregacion expresa que un objeto agrupa a otros que pueden existir de forma independiente. Catalogo agrupa un arreglo de Libro[]. Los libros existian antes de ser agregados al catalogo y seguirian existiendo si el catalogo desapareciera.

Igualmente, GestorPrestamos mantiene un arreglo de Prestamo[]. Los objetos Libro y Usuario dentro de cada Prestamo son externos al gestor; el solo los referencia.

Fragmento clave: Catalogo.java

```

public class Catalogo {

    // AGREGACION: los libros existen fuera del catalogo
    private Libro[] libros;
    private int     cantidad;

    // Solo almacena la referencia; no crea ni destruye los libros
    public void agregarLibro(Libro libro) {
        libros[cantidad] = libro;
        cantidad++;
    }
}

```

2.6 Dependencia

La dependencia es la relacion mas debil: un metodo recibe un objeto como parametro, lo usa puntualmente y no lo guarda. En prestarLibro(), el isbn y el usuario llegan como argumentos. GestorPrestamos no los almacena como atributos propios; solo los usa para crear el Prestamo en ese momento.

Fragmento clave: GestorPrestamos.java

```

// DEPENDENCIA: isbn y usuario son parametros, no atributos del gestor
public void prestarLibro(String isbn, Usuario usuario, String fecha) {
    Libro libro = catalogo.buscarPorIsbn(isbn);
    if (libro == null || !libro.isDisponible()) { /* manejo de error */
        return;
    }
    Prestamo p = new Prestamo(libro, usuario, fecha);
    prestamos[numPrestamos++] = p;
}

```

3. Diagrama UML e interpretacion

Descripcion del diagrama

El diagrama de clases muestra siete clases con sus atributos, metodos y relaciones. Las convenciones utilizadas son:

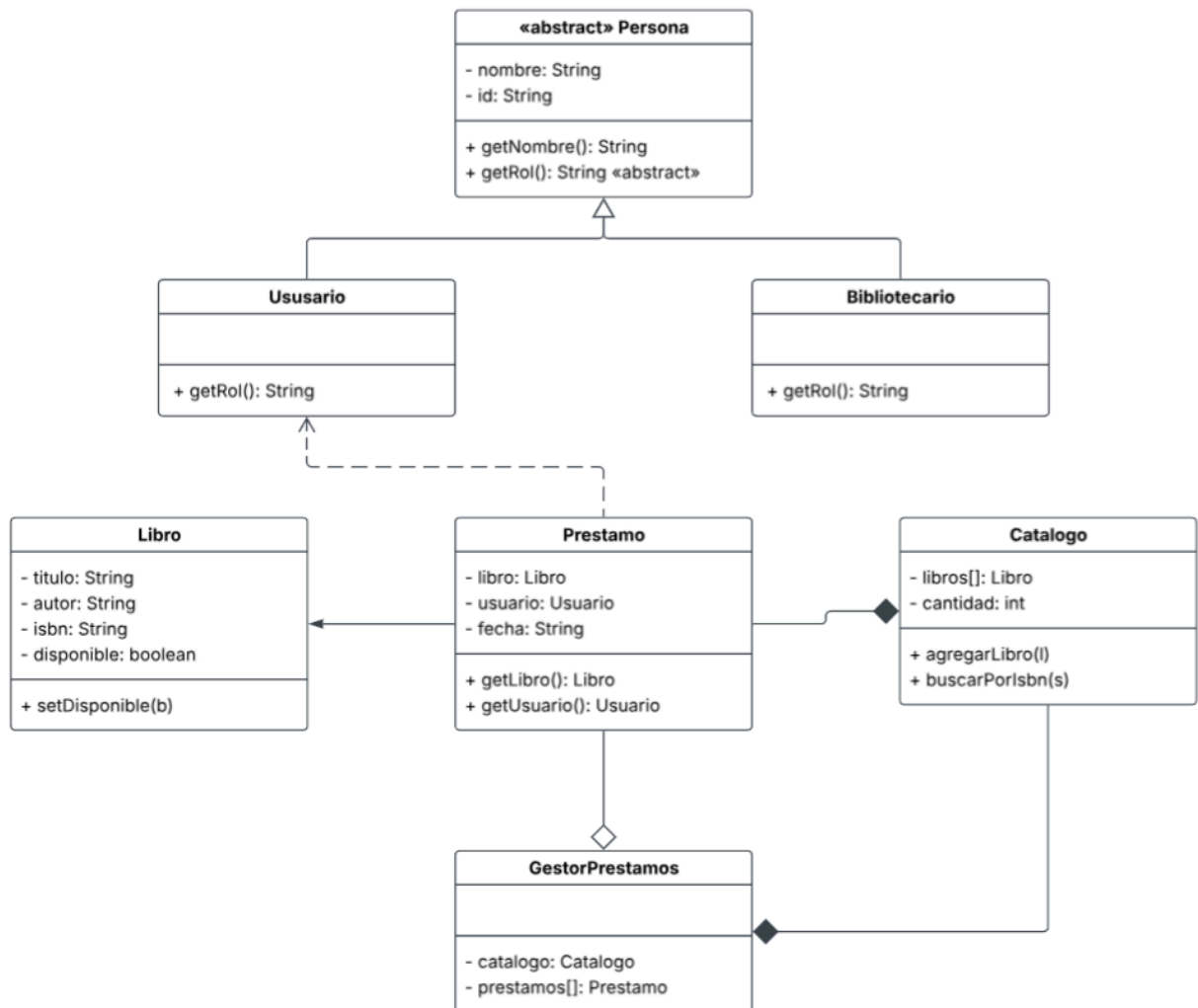
- Flecha triangular hueca (herencia): la subclase extiende a la superclase.
- Rombo relleno (composicion): el objeto contenedor crea y posee al contenido.
- Rombo hueco (agregacion): el objeto referencia a otros que existen independientemente.
- Flecha punteada (dependencia): un metodo recibe el objeto como parametro temporal.

Lectura del diagrama

Persona es la raiz de la jerarquia. Usuario y Bibliotecario heredan de ella. Prestamo tiene composicion con Libro y Usuario: los necesita para existir. Catalogo agrega Libros mediante un arreglo. GestorPrestamos tiene composicion con Catalogo (lo crea) y agrega Prestamos (los registra). La dependencia entre prestarLibro() y Usuario/isbn es la relacion mas efimera: dura solo lo que tarda la llamada al metodo.

El diagrama fue construido con las convenciones estandar UML y puede verse en la seccion visual del portafolio.

Diagrama UML



4. Reflexion personal

Desafios enfrentados

El primer desafio fue distinguir entre composicion y agregacion. Al principio parecia una distincion teorica sin impacto real, pero en la implementacion la diferencia es concreta: si GestorPrestamos crea su Catalogo en el constructor (composicion), no puede recibir otro catalogo desde afuera. Si Catalogo recibe Libros por parametro (agregacion), esos libros pueden actualizarse desde Main sin que el catalogo lo sepa. Entender esto cambio como pense el diseno desde el principio.

El segundo desafio fue el encapsulamiento real. Es tentador hacer todo public para que el codigo 'funcione rapido', pero al modelar Libro con disponible como private, el sistema obliga a pensar: quien tiene derecho a cambiar este estado? La respuesta es el Prestamo, y solo el Prestamo. Esa restriccion no es un obstaculo: es el sistema diciendo donde vive la logica de negocio.

Aprendizajes obtenidos

La abstraccion no es solo hacer clases con herencia: es decidir que ignorar. Persona ignora si alguien tiene cuenta de correo, tarjeta de credito o cualquier otro dato que el sistema de biblioteca no necesita. Esa decision de ignorar es tan importante como la decision de incluir.

El encapsulamiento no es burocracia de getters y setters. Es la diferencia entre un sistema donde cualquier parte puede corromper el estado de cualquier objeto, y un sistema donde cada objeto es responsable de su propio estado.

Importancia de cada principio en el desarrollo de software

Principio	Impacto en software real
Abstraccion	Permite cambiar implementacion sin afectar quien la usa
Encapsulamiento	Reduce superficie de bugs; el estado solo cambia desde puntos conocidos
Herencia	Elimina duplicacion de codigo; un cambio en Persona afecta a todos los roles
Composicion	Garantiza coherencia: Prestamo nunca puede quedar huero de Libro
Agregacion	Permite reutilizar objetos (un Libro en multiples catalogos futuros)
Dependencia	Mantiene bajo el acoplamiento; los metodos no acumulan referencias innecesarias

En conjunto, estos principios no son reglas academicas: son la razon por la que un sistema puede crecer de 7 clases a 70 sin convertirse en un caos. La POO bien aplicada hace que cada nueva funcionalidad encuentre el lugar donde naturalmente pertenece.